The block inside the shared_examples method is similar to a describe block. Inside, you can use let to define variables or it to define specs. The only unusual thing about this block is that rather than create an object explicitly, the let statement at the top creates a generic instance using the described_class, which is the class referenced by the innermost describe whose argument is a class. Most of the time this will be the initial RSpec.describe. We're also "setting" the size via a mock on the theory that while tasks have a setter for their size, projects don't.

To use the shared example group, all you need to do is declare it. RSpec defines multiple equivalent methods for doing so (and even allows you to define your own), one of which is it_should_behave_like followed by the name of the group. Here's what that looks like in the task_spec file:

model/01/spec/models/task_spec.rb
```
RSpec.describe Task do

  it_should_behave_like "sizeable"
```

When task_spec is executed, the shared group is invoked with Task as the described_class, and the test will look for the small? and epic? methods to pass the test. To make the code pass, you need to add the concern:

model/01/app/models/concerns/sizeable.rb
```
module Sizeable
  extend ActiveSupport::Concern

  def small?
    size <= 1
  end

  def epic?
    size >= 5
  end
end
```

And then include it in the Task class:

model/01/app/models/task.rb
```
class Task < ApplicationRecord
  include Sizeable
```

With that in place you can quickly add both the spec and the feature to the project class (assuming you rename the project method from total_size to size):

model/01/spec/models/project_spec.rb
```
RSpec.describe Project do

  it_should_behave_like "sizeable"
```

and